arXiv:2603.04334v1 [cs.DB] 4 Mar 2026

SpotIt$^+$: Verification-based Text-to-SQL Evaluation with Database Constraints

Andrew Tremante[1], Yang He[2], Rocky Klopfenstein[1], Yuepeng Wang[2], Nina Narodytska[3], and Haoze Wu[1,3]

[1] Amherst College
{atremante26,rklopfenstein27,hwu}@amherst.edu
[2] Simon Fraser University
{yha244,yuepeng}@sfu.ca
[3] VMware Research by Broadcom
nina.narodytska@broadcom.com

Abstract. We present SpotIt$^+$, an open-source tool for evaluating Text-to-SQL systems via bounded equivalence verification. Given a generated SQL query and the ground truth, SpotIt$^+$ actively searches for database instances that differentiate the two queries. To ensure that the generated counterexamples reflect practically relevant discrepancies, we introduce a constraint-mining pipeline that combines rule-based specification mining over example databases with LLM-based validation. Experimental results on the BIRD dataset show that the mined constraints enable SpotIt$^+$ to generate more realistic differentiating databases, while preserving its ability to efficiently uncover numerous discrepancies between generated and gold SQL queries that are missed by standard test-based evaluation.

1 Introduction

Text-to-SQL – the task of translating natural language questions into executable database queries – constitutes a key building block of modern chatbots and intelligent assistants across a wide range of industrial applications [1,2,3,18]. To compare and track the capabilities of current Text-to-SQL methods, several community-driven evaluation platforms have been proposed [12,15]. These platforms are widely endorsed by both industry and academia as primary benchmarks for assessing the state of the art in Text-to-SQL.

Determining whether a Text-to-SQL method produces a correct SQL query is non-trivial. Existing platforms predominantly rely on *test-based evaluation*: executing the generated SQL query and the human-labeled ground truth (i.e., the gold SQL) on a fixed test database and comparing the results. This approach can be overly optimistic, as two non-equivalent queries may return identical results on test database instances. Recent work [11] demonstrates that test-based evaluation does frequently overlook such discrepancies and proposes a verification-based alternative that provides stronger correctness guarantees. The key idea is to use an SMT-based bounded verification engine [10] to systematically search for database instances that differentiate the generated and gold queries. The outcome

is either a proof of equivalence within the bounded search space, or a concrete counterexample database witnessing non-equivalence.

A natural concern with verification-based evaluation is the practical relevance of the discovered counterexamples. In addition to explicitly stated integrity constraints (e.g., primary keys and foreign key relationships), databases often contain domain-specific constraints that are implicit yet should be obeyed from a practical standpoint. With these constraints unexposed to the underlying verifier, the found counterexample databases might reveal discrepancies that only occur in pathological corner cases that are impossible or unlikely to arise in practice. Although the ultimate assessment of realism rests with human judgment, we argue that a practical Text-to-SQL evaluation framework should strive to ensure that counterexamples reflect plausible data distributions whenever possible.

In this paper, we present SpotIt$^+$, a bounded-verification-based tool for Text-to-SQL evaluation. SpotIt$^+$ goes beyond an open-source implementation of the verification-based pipeline described in prior work [11]. It features a constraint-extraction pipeline that mines database constraints from example databases. These constraints are encoded as SMT constraints during bounded verification. Purely rule-based constraint mining, however, risks overfitting to idiosyncrasies of the example databases. To mitigate this risk, SpotIt$^+$ incorporates a large language model (LLM) to assess whether a mined constraint represents a genuine domain property before including it in the verification query.

We employ SpotIt$^+$ to evaluate ten state-of-the-art Text-to-SQL methods on the popular BIRD dataset [15]. Our results show that incorporating the extracted database constraints yields more realistic counterexamples, while still uncovering a substantial number of discrepancies that test-based evaluation overlooks. To summarize, our contributions include:

- SpotIt$^+$, an open-source verification-based tool[4] for Text-to-SQL evaluation;
- A constraint-extraction pipeline that combines rule-based specification mining with LLM-based validation and repair;
- An empirical evaluation demonstrating that SpotIt$^+$ improves the realism of counterexamples while preserving strong discrepancy-detection capability.

2 Motivating Example

Before presenting the workflow of SpotIt$^+$, we first illustrate the effect of constraint extraction and LLM validation through a motivating example. Consider the following question from the BIRD dataset [15]: *Please list the account types that are not eligible for loans, and the average income of residents in the district where the account is located exceeds $8000 but is no more than $9000.* Consider the generated query and the gold query for this question in Figure 1.

These queries differ in how they filter the column `DISTRICT.A11`. The generated query includes only values strictly greater than 8000 (`DISTRICT.A11 >`

[4] Available at `https://github.com/atremante26/SpotItPlus`

```sql
/* Generated Query */
SELECT DISTINCT DISP.TYPE
FROM DISP
INNER JOIN ACCOUNT
  ON DISP.ACCOUNT_ID = ACCOUNT.
       ACCOUNT_ID
INNER JOIN DISTRICT
  ON ACCOUNT.DISTRICT_ID = DISTRICT.
       DISTRICT_ID
WHERE DISP.TYPE <> 'OWNER'
  AND DISTRICT.A11 > 8000
  AND DISTRICT.A11 <= 9000;
```

```sql
/* Gold Query */
SELECT T3.TYPE
FROM DISTRICT AS T1
INNER JOIN ACCOUNT AS T2
  ON T1.DISTRICT_ID = T2.
       DISTRICT_ID
INNER JOIN DISP AS T3
  ON T2.ACCOUNT_ID = T3.
       ACCOUNT_ID
WHERE T3.TYPE != 'OWNER'
  AND T1.A11 BETWEEN 8000 AND
       9000;
```

Fig. 1: The generated and gold queries for a question in the BIRD dataset.

8000), while the gold query uses `BETWEEN 8000 AND 9000`, which includes 8000.[5] The official test-based evaluation incorrectly deems these two queries equivalent because they produce the same result on the test database. In contrast, bounded verification without extracted database constraints finds the following counterexample that distinguishes the two queries:

```sql
--Vanilla Counterexample--
CREATE TABLE DISTRICT (DISTRICT_ID INTEGER, A2 VARCHAR(20), A3 VARCHAR
    (20), ..., A11 INTEGER, ...);
INSERT INTO DISTRICT VALUES (0, '2147483648', '2147483648', ..., 8000,
    ...);
CREATE TABLE ACCOUNT (ACCOUNT_ID INTEGER, DISTRICT_ID INTEGER, FREQUENCY
    VARCHAR(20), ...);
INSERT INTO ACCOUNT VALUES (0, 0, '2147483648', ...);
INSERT INTO ACCOUNT VALUES (1, 0, '2147483648', ...);
CREATE TABLE DISP (DISP_ID INTEGER, CLIENT_ID INTEGER, ACCOUNT_ID INTEGER
    , TYPE VARCHAR(20));
INSERT INTO DISP VALUES (0, 0, 0, '2147483648');
INSERT INTO DISP VALUES (1, 0, 1, '5565851591214670 50');
```

The counterexample generated by SpotIt$^+$ with analytically extracted and LLM-validated database constraints is as follows:

```sql
--LLM-Validated Counterexample--
CREATE TABLE DISTRICT (DISTRICT_ID INTEGER, A2 VARCHAR(20), A3 VARCHAR
    (20), ..., A11 INTEGER, ...);
INSERT INTO DISTRICT VALUES (1, '2147483648', 'Prague', ..., 8000, ...);
CREATE TABLE ACCOUNT (ACCOUNT_ID INTEGER, DISTRICT_ID INTEGER, FREQUENCY
    VARCHAR(20), ...);
INSERT INTO ACCOUNT VALUES (1, 1, 'POPLATEK TYDNE', ...);
INSERT INTO ACCOUNT VALUES (2, 1, 'POPLATEK PO OBRATU', ...);
CREATE TABLE DISP (DISP_ID INTEGER, CLIENT_ID INTEGER, ACCOUNT_ID INTEGER
    , TYPE VARCHAR(20));
INSERT INTO DISP VALUES (1, 1, 1, 'DISPONENT');
INSERT INTO DISP VALUES (2, 1, 1, 'DISPONENT');
```

The counterexample generated by SpotIt$^+$ is qualitatively more realistic. In particular, categorical constraints were extracted for columns such as `DISTRICT.A2`, `ACCOUNT.FREQUENCY`, and `DISP.TYPE`, which state that the column values must belong to one of several fixed choices.

On the other hand, if we add all constraints mined directly from the test database without LLM-based validation, bounded verification fails to find a

[5] In this case, the gold SQL is incorrect, which is a separate issue discussed in prior work [11].

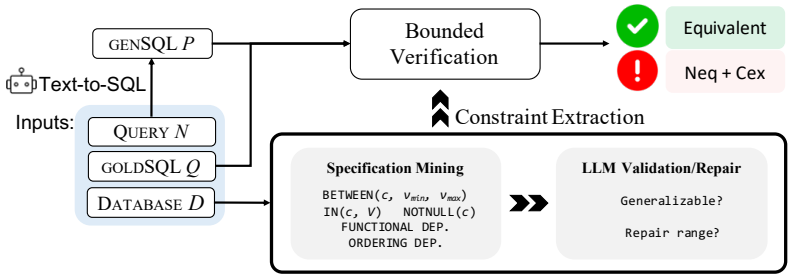


Fig. 2: The workflow of SPOTIT$^+$.

counterexample in this case. This is because, in the test database, the value of `DISTRICT.A11` ranges from 8110 to 12541 and does not include the critical boundary value 8000. Adding this range constraint would eliminate any potential counterexample. Clearly, this constraint is overly restrictive, and the LLM validation pass successfully relaxes it.

3 SpotIt$^+$: Bounded Equivalence Verification with Database Constraints

The workflow of SPOTIT$^+$ is shown in Fig. 2. A Text-to-SQL benchmark consists of a natural language question N, the gold SQL Q, and an example database D. A Text-to-SQL method takes as input N and generates a SQL query P. The goal of SPOTIT$^+$ is to determine whether P should be deemed a correct prediction, and it does so by checking its bounded equivalence with the gold SQL Q under database constraints. Unlike previous work [11] that only considers integrity constraints explicitly stated in the database schema, SPOTIT$^+$ features a constraint extraction module which takes as input the example database D, mines constraints from it, and validates/repairs the constraints using an LLM. Those constraints are encoded in the bounded verification query to rule out unrealistic counterexamples. Background on bounded SQL equivalence checking can be found in App. A. Here, we focus on the constraint extraction module.

3.1 Constraint Types

Currently, SPOTIT+ extracts five types of constraints from the example databases.

Range Constraints. RANGE constraints restrict a numeric column to values within a specified interval:

$$\textsc{Between}(c, v_{\min}, v_{\max}) \triangleq \forall i.\ v_{\min} \leq c[i] \leq v_{\max}$$

where c is a column, $c[i]$ denotes the value of column c at row i, and $v_{\min}$ and $v_{\max}$ define the lower and upper bounds.

Categorical Constraints. CATEGORICAL constraints restrict a column to a finite set of discrete values:

$$\textsc{In}(c, V) \triangleq \forall i.\ c[i] \in V$$

where $V = \{v_1, v_2, \ldots, v_k\}$ is a finite set of allowed values.

NotNull Constraints. NotNull constraints specify that a column cannot contain null values:

$$\textsc{NotNull}(c) \triangleq \forall i.\ c[i] \neq \text{Null}$$

Functional Dependencies. Functional Dependencies state that one set of columns uniquely determines another set:

$$X \to Y \triangleq \forall i, j.\ X[i] = X[j] \implies Y[i] = Y[j]$$

where X and Y are sets of columns, and i, j range over all pairs of rows.

Ordering Dependencies. Ordering Dependencies enforce an inequality relationship between two numeric columns:

$$c_1 \lhd c_2 \triangleq \forall i.\ c_1[i] \lhd c_2[i]$$

where c_1 and c_2 are numeric columns, $\lhd \in \{\leq, \geq\}$ is a comparison operator.

3.2 Constraint Extraction Pipeline

SpotIt$^+$ takes as input an example database (which in reality is provided as part of the official test-based evaluation) and automatically extracts the five types of constraints on all eligible columns in the database. Range constraints are computed for each column of numerical type by computing the min and max values in the database. Categorical constraints are extracted for all columns with at most K unique values, where each unique value is considered to be a category. K is a configurable parameter and is by default set to 30. NotNull constraints are generated for columns without nulls in the test database. Functional Dependencies are extracted for all non-primary column pairs c_1, c_2 that satisfy the functional dependency relation. Ordering Dependencies are extracted for non-primary, numerical column pairs c_1, c_2 that satisfy an inequality relation.

Next, SpotIt$^+$ uses an LLM to filter the inferred candidate constraints and repair over-constraining range constraints. Specifically, for each extracted constraint, a prompt that asks the LLM to evaluate whether the constraint holds beyond the test database is formulated. For example, a functional dependency between `country code` and `country name` represents a genuine relationship, while a dependency between `firstname` and `occupation` does not. For numerical columns, the LLM is further prompted to judge whether the extracted range constraint is too restrictive and should be repaired. For example, suppose the test database only contains `patient age` ranging from 30 to 60. The LLM may suggest to change the range constraint to $[0, 120]$. Note that it is ultimately up to the Text-to-SQL/database user to decide which constraints should be included, and our goal is to provide configurable levels of strictness for equivalence checking and allow users to choose the most appropriate for their needs.

The extracted database constraints are then encoded in the bounded verification engine to focus the search on practically relevant data distribution. App. B provides additional implementation details on the constraint extraction module.

4 Evaluation

We use SpotIt$^+$ to evaluate Text-to-SQL methods on the popular BIRD dataset [15], whose publicly available dev-set contains 1,533 questions across 11 databases spanning healthcare, education, and other professional domains. We obtained predictions from 10 state-of-the-art Text-to-SQL methods (shown in Table 1) on the BIRD leaderboard [6]. We are primarily interested in the following research questions:

Table 1: The considered methods.

Entry	Acronym
Alpha-SQL + Qwen2.5-Coder-32B [13]	Alpha
CSC-SQL + Qwen2.5-Coder-7B [16]	csc-7b
CSC-SQL + XiYanSQL [16]	csc-32b
GenaSQL-1 [9]	gena-1
GenaSQL-2 [9]	gena-2
RSL-SQL + GPT-4o [4]	rsl
OmniSQL-32B[14]	omni-maj
GSR (anonymous authors)	gsr
CHESS$_{\text{IR+CG+UT}}$ [19]	Chess
SLM-SQL + Qwen2.5-Coder-1.5B [17]	slm

- How do the accuracy measurements change when we check SQL equivalence under additional database constraints?
- Can SpotIt$^+$ efficiently evaluate prediction correctness with the additional database constraints?
- Do the database constraints lead to more realistic counterexamples?

We first evaluate the predictions of each method using BIRD's official test-based execution accuracy metric (EX-test), which compares the results of executing the generated and gold queries on a given test database. We then apply SpotIt$^+$ to predictions that are deemed correct by EX-test. We consider three configurations: 1) SpotIt: bounded equivalence checking without extracted database constraints. This is the configuration used in prior work [11]; 2) SpotIt$^+$-noV: bounded equivalence checking with extracted database constraints, but without LLM validation and repair; and 3) SpotIt$^+$: bounded verification with LLM-validated constraints. We verify each SQL pair up to a bound of 5 rows per table. Each pair is assigned one physical core, 8GB of memory, and a CPU timeout of 600 seconds. In practice, counterexamples are typically found within seconds, as discussed below. Experiments were conducted on a cluster of Dell PowerEdge R6525 servers equipped with 2.6GHz AMD CPU cores.

4.1 Verification Results

Table 2 reports the accuracy of each Text-to-SQL method under the official test-based metric EX-test and the three verification-based evaluation metrics. All verification-based configurations identify substantially more discrepancies between generated and gold SQL queries than the test-based evaluation.

Compared to SpotIt, both SpotIt$^+$-noV and SpotIt$^+$ deem fewer query pairs inequivalent. This is expected as the extracted database constraints eliminate query discrepancies that arise only in unrealistic database instances. Interestingly, comparing SpotIt$^+$-noV and SpotIt$^+$ shows that LLM validation does not lead

[6] `https://bird-bench.github.io/`

Table 2: Performance of Text-to-SQL methods using EX-TEST, SPOTIT, SPOTIT$^+$-NOV, and SPOTIT$^+$ on the 1533 BIRD-dev benchmarks.

	EX-test		**EX-SpotIt**		**EX-SpotIt$^+$-noV**		**EX-SpotIt$^+$**	
	Acc. (%)	Rnk	Acc. (%)	Rnk	Acc. (%)	Rnk	Acc. (%)	Rnk
CSC-32B	71.32	1	60.10	3	63.56	2	63.43	2
GENA-2	70.53	2	60.89	1	63.75	1	63.56	1
ALPHA	69.36	3	56.91	6	61.21	5	60.56	5
GENA-1	69.23	4	60.50	2	63.23	3	63.17	3
CSC-7B	69.17	5	59.91	4	62.58	4	62.26	4
RSL	67.67	6	57.63	5	60.82	6	60.56	6
OMNI-MAJ	66.88	7	55.80	8	59.52	7	59.32	7
GSR	66.49	8	55.93	7	59.13	8	58.87	8
CHESS	63.62	9	53.91	9	56.91	9	56.45	9
SLM	63.43	10	52.54	10	56.00	10	55.54	10

to a substantial change in the number of pairs deemed inequivalent. We hypothesize that most database constraints do not eliminate counterexamples outright, but only make them more realistic. Nonetheless, as shown in Sec. 2, LLM-based validation does recover genuine discrepancies from overly restrictive constraints sometimes. Appendix D provides examples where SPOTIT$^+$ rules out counterexamples found by SPOTIT, and additional examples illustrating differences between counterexamples generated with and without database constraints.

The average time to find a counterexample is 1.7 seconds for SPOTIT, 1.4 seconds for SPOTIT$^+$-NOV, and 0.9 seconds for SPOTIT$^+$, indicating that SPOTIT$^+$ is practically efficient. A detailed runtime breakdown is provided in Appendix C. Currently, the bounded equivalence verification engine (VeriEQL [10]) successfully encodes 93–97% of the examined SQL pairs across the ten Text-to-SQL methods. While the coverage on BIRD benchmarks is already high, closing the remaining gap is an important next step.

5 Conclusion

We presented SPOTIT$^+$, an open-source Text-to-SQL evaluation tool based on bounded equivalence verification and database constraint extraction. Currently, SPOTIT$^+$ extracts from example databases five types of constraints and uses an LLM to validate and repair the extracted constraints. Experimental evaluation demonstrated that SPOTIT$^+$ is readily applicable to high-profile Text-to-SQL evaluation platform such as BIRD, capable of generating realistic differentiating databases, and effective at uncovering discrepancies undetected by the official test-based evaluation. Future work includes designing richer constraint extraction to support cross-table relationships, extending the underlying bounded equivalence checker to larger SQL fragments, incorporating user-specified domain knowledge alongside automatic extraction, and investigating constraint-enhanced verification for SQL verification tasks beyond Text-to-SQL evaluation.

References

1. Amazon: Build a text-to-sql solution for data consistency in generative ai using amazon nova (September 2025), https://aws.amazon.com/blogs/machine-learning/build-a-text-to-sql-solution-for-data-consistency-in-generative-ai-using-amazon-nova/
2. Amazon Web Services: How msd uses amazon bedrock to translate natural language into sql for complex healthcare databases. https://aws.amazon.com/blogs/machine-learning/how-merck-uses-amazon-bedrock-to-translate-natural-language-into-sql-for-complex-healthcare-databases/ (2024), accessed: 2025-09-21
3. BitsAI, T.: Bits ai:scale your teams with autonomous ai agents across monitoring, development, and security (September 2025), https://www.datadoghq.com/product/platform/bits-ai/
4. Cao, Z., Zheng, Y., Fan, Z., Zhang, X., Chen, W., Bai, X.: Rsl-sql: Robust schema linking in text-to-sql generation. arXiv preprint arXiv:2411.00073 (2024)
5. Chu, S., Li, D., Wang, C., Cheung, A., Suciu, D.: Demonstration of the cosette automated sql prover. In: Proceedings of the 2017 ACM International Conference on Management of Data. pp. 1591–1594 (2017)
6. Chu, S., Murphy, B., Roesch, J., Cheung, A., Suciu, D.: Axiomatic foundations and algorithms for deciding semantic equivalences of sql queries. arXiv preprint arXiv:1802.02229 (2018)
7. Chu, S., Wang, C., Weitz, K., Cheung, A.: Cosette: An automated prover for sql. In: CIDR. pp. 1–7 (2017)
8. Chu, S., Weitz, K., Cheung, A., Suciu, D.: Hottsql: Proving query rewrites with univalent sql semantics. Acm sigplan notices **52**(6), 510–524 (2017)
9. Dönder, Y.D., Hommel, D., Wen-Yi, A.W., Mimno, D., Jo, U.E.S.: Cheaper, better, faster, stronger: Robust text-to-sql without chain-of-thought or fine-tuning. arXiv preprint arXiv:2505.14174 (2025)
10. He, Y., Zhao, P., Wang, X., Wang, Y.: Verieql: Bounded equivalence verification for complex sql queries with integrity constraints. Proc. ACM Program. Lang. **8**(OOPSLA1) (Apr 2024). https://doi.org/10.1145/3649849, https://doi.org/10.1145/3649849
11. Klopfenstein, R., He, Y., Tremante, A., Wang, Y., Narodytska, N., Wu, H.: Spotit: Evaluating text-to-sql evaluation with formal verification. In: The Fourteenth International Conference on Learning Representations (2026), https://openreview.net/forum?id=iMkvR2ICSE
12. Lei, F., Chen, J., Ye, Y., Cao, R., Shin, D., Su, H., Suo, Z., Gao, H., Hu, W., Yin, P., et al.: Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. arXiv preprint arXiv:2411.07763 (2024)
13. Li, B., Zhang, J., Fan, J., Xu, Y., Chen, C., Tang, N., Luo, Y.: Alpha-sql: Zero-shot text-to-sql using monte carlo tree search. arXiv preprint arXiv:2502.17248 (2025)
14. Li, H., Wu, S., Zhang, X., Huang, X., Zhang, J., Jiang, F., Wang, S., Zhang, T., Chen, J., Shi, R., et al.: Omnisql: Synthesizing high-quality text-to-sql data at scale. arXiv preprint arXiv:2503.02240 (2025)
15. Li, J., Hui, B., Qu, G., Li, B., Yang, J., Li, B., Wang, B., Qin, B., Cao, R., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K.C., Huang, F., Cheng, R., Li, Y.: BIRD_SQ. https://bird-bench.github.io/ (2024), accessed: Sep 2025
16. Sheng, L., Xu, S.S.: Csc-sql: Corrective self-consistency in text-to-sql via reinforcement learning. arXiv preprint arXiv:2505.13271 (2025)

17. Sheng, L., Xu, S.S.: Slm-sql: An exploration of small language models for text-to-sql. arXiv preprint arXiv:2507.22478 (2025)
18. Splunk: Ai assistant in observability cloud (September 2025), https://www.splunk.com/en_us/products/splunk-ai-assistant-in-observability-cloud.html
19. Talaei, S., Pourreza, M., Chang, Y.C., Mirhoseini, A., Saberi, A.: Chess: Contextual harnessing for efficient sql synthesis. arXiv preprint arXiv:2405.16755 (2024)
20. Veanes, M., Tillmann, N., De Halleux, J.: Qex: Symbolic sql query explorer. In: International Conference on Logic for Programming Artificial Intelligence and Reasoning. pp. 425–446. Springer (2010)
21. Wang, S., Pan, S., Cheung, A.: Qed: A powerful query equivalence decider for sql. Proceedings of the VLDB Endowment **17**(11), 3602–3614 (2024)
22. Zhou, A., Wang, K., Lu, Z., Shi, W., Luo, S., Qin, Z., Lu, S., Jia, A., Song, L., Zhan, M., Li, H.: Solving challenging math word problems using GPT-4 code interpreter with code-based self-verification. In: The Twelfth International Conference on Learning Representations (2024), https://openreview.net/forum?id=c8McWs4Av0
23. Zhou, Q., Arulraj, J., Navathe, S.B., Harris, W., Wu, J.: Spes: A symbolic approach to proving query equivalence under bag semantics. In: 2022 IEEE 38th International Conference on Data Engineering (ICDE). pp. 2735–2748. IEEE (2022)

A Background

Determining whether two SQL queries produce identical results on all possible database instances is generally unsolvable for expressive SQL fragments [6,8,21,22,23]. Bounded equivalence verification addresses this by checking equivalence within a finite search space: given queries Q_1 and Q_2 over schema S and bound K, the goal is to determine whether Q_1 and Q_2 are equivalent on all databases where each relation has at most K tuples [5,7,10,20]. Formally:

$$Q_1 \simeq_{S,K} Q_2 \triangleq \forall D \in \text{Instances}(S).\ \forall R \in \text{Relations}(D).\\ |R| \leq K \implies Q_1(D) = Q_2(D)$$

Bounded verification either proves that equivalence holds for all databases up to size K, or produces a concrete counterexample database D_{cex} demonstrating $Q_1(D_{\text{cex}}) \neq Q_2(D_{\text{cex}})$.

A.1 VeriEQL

VeriEQL [10] is a bounded equivalence checker that reduces SQL verification to satisfiability modulo theories (SMT) solving. Given queries Q_1 and Q_2 over schema S and bound K, VeriEQL symbolically executes both queries on a database with K symbolic tuples per relation, encoding the execution and non-equivalence condition as an SMT formula solvable by Z3.

The resulting formula takes the form $\Phi = \Phi_C \wedge \Phi_{Q_1} \wedge \Phi_{Q_2} \wedge \neg\Phi_{\Leftrightarrow}$ where Φ_C encodes integrity constraints, Φ_{Q_1} and Φ_{Q_2} encode the semantics of each query, and $\neg\Phi_{\Leftrightarrow}$ asserts that the two query outputs differ.

For instance, to verify `SELECT id FROM R WHERE id > 1` against `SELECT id FROM R WHERE id > 2` with $K = 1$, VeriEQL creates symbolic tuple $t_1 = [x_1, x_2]$ for relation R and result tuples $Q_1(D) = [t_2]$ where $t_2 = [x_3]$ and $Q_2(D) = [t_3]$ where $t_3 = [x_4]$ for each query. Query semantics are captured through constraints such as:

$$\Phi_{Q_1} = (x_1 > 1 \rightarrow (x_3 = x_1 \wedge \neg\text{Del}(t_2))) \wedge (x_1 \leq 1 \rightarrow \text{Del}(t_2))$$

where Del is a deletion flag indicating tuple non-existence. The conjunction $\Phi_{Q_1} \wedge \Phi_{Q_2} \wedge (t_2 \neq t_3)$ is satisfiable if and only if a differentiating database exists.

VeriEQL supports a rich SQL subset including joins, aggregations, subqueries, set operations, and user-defined integrity constraints such as primary keys, foreign keys, and uniqueness constraints.

B SpotIt$^+$

B.1 Specification Mining

Given a database instance D over schema S, we extract candidate constraints from the observed data for each of the constraint types.

Range Constraints. For numeric columns with at least 2 distinct values, we derive bounds using three complementary approaches:

1. **Strict bounds** use observed minimum and maximum values $[\min(c), \max(c)]$, ensuring every current database value satisfies the constraint.
2. **Loose bounds** apply Tukey's fence method ($k = 3$): $[Q_1 - 3 \cdot \text{IQR},\ Q_3 + 3 \cdot \text{IQR}]$, accommodating outliers while excluding unrealistic extremes. For columns where observed minimum is ≥ 0, we enforce a lower bound of 0.
3. **LLM-generated semantic bounds** infer domain-appropriate ranges from column metadata, observed statistics, and sample values (e.g., ages $[0, 120]$, percentages $[0, 100]$, one-sided bounds $[0, \infty)$ for counts).

During validation, each range constraint is reviewed and one of these three bounding approaches is selected. In our experiments, the LLM validator chose between strict and semantic bounds.

Categorical Constraints. For columns with 2-30 unique values:

1. Extract the complete set of observed values $V = \{v_1, \ldots, v_k\}$.
2. Generate a candidate constraint: $\textsc{In}(c, V)$.

Columns with cardinality > 30 are automatically excluded to prevent over-constraining the counterexample search space.

Null Constraints. For columns with zero observed null values, we generate a candidate constraint. The process of filtering primary key constraints occurs during validation.

Functional Dependencies. For each pair of columns (c_a, c_b):

1. Compute $n_a = |\text{distinct}(c_a)|$ and $n_{ab} = |\text{distinct}(c_a, c_b)|$.
2. Verify that grouping by c_a produces groups with exactly one unique c_b value each.
3. Generate a candidate constraint: $c_a \rightarrow c_b$.

Ordering Dependencies. For each pair of numeric columns (c_1, c_2):

1. Test whether $c_1[i] \leq c_2[i]$ holds for all non-null rows.
2. Test whether $c_1[i] \geq c_2[i]$ holds for all non-null rows.
3. If either relationship holds, generate a candidate constraint.

B.2 Implementation

Our extraction system is implemented in Python 3.11 and consists of approximately 2,300 lines of code organized into five main modules:

– `extract_constraints_LLM.py` (∼800 lines): Orchestrates the SpotIt$^+$ constrain extraction pipeline.

Table 3: Average and median runtime (in seconds) for counterexample generation across Text-to-SQL methods. Δ columns show the difference from SpotIt (positive values indicate slower performance).

	SpotIt		**SpotIt$^+$-noV**			**SpotIt$^+$**		
Method	Avg.	Med.	Avg.	Med.	ΔAvg.	Avg.	Med.	ΔAvg.
csc-32b	2.87	0.33	0.72	0.46	-2.15	0.85	0.47	-2.02
gena-2	1.06	0.30	2.26	0.36	+1.20	0.98	0.34	-0.08
Alpha	3.67	0.36	0.99	0.58	-2.68	1.57	0.55	-2.10
gena-1	0.82	0.39	0.66	0.40	-0.16	0.57	0.37	-0.25
csc-7b	0.58	0.29	3.08	0.36	+2.50	1.37	0.34	+0.79
rsl	3.34	0.41	1.24	0.48	-2.10	0.90	0.43	-2.44
omni-maj	0.94	0.35	2.56	0.59	+1.62	0.65	0.50	-0.29
gsr	0.80	0.34	0.75	0.53	-0.05	0.85	0.45	+0.05
Chess	1.26	0.28	0.75	0.41	-0.51	0.67	0.39	-0.59
slm	1.29	0.29	0.66	0.36	-0.63	0.56	0.37	-0.73
Mean	1.66	0.33	1.37	0.45	-0.30	0.90	0.42	-0.77

- `extract_constraints.py` (∼800 lines): Orchestrates the SpotIt$^+$-noV constraint extraction pipeline.
- `utils.py` (∼100 lines): Dependency detection and schema mapping utilities.
- `llm.py` (∼300 lines): Language model interaction and prompt management using OpenAI's API.
- `prompts.py` (∼600 lines): Structured prompts for semantic bound generation and validation tasks.

The system uses OpenAI's *gpt-5.1* model with JSON mode enabled for parsable outputs.

C Experimental Evaluation

C.1 Verification Runtime

Table 3 presents runtime statistics for counterexample generation. For all Text-to-SQL methods and approaches, runtime performance remains efficient. SpotIt$^+$-noV constraints decrease average runtime by 0.3 seconds compared to SpotIt. SpotIt$^+$ constraints decrease average runtime further by 0.77 seconds. Median runtime remain below 0.6 seconds across all approaches, indicating that the majority of queries are verified efficiently. These improvements are attributable to the restricted search space imposed by the additional constraints.

D Additional Counterexamples

We show additional examples illustrating the effect of considering database constraints in bounded verification. The first two examples are cases where the database constraints ruled out unrealistic counterexamples. The next two examples show the qualitative differences between counterexamples generated with and without database constraints.

Example D.1. Consider the natural language question: *"How much is the average build up play speed of the Heart of Midlothian team?"*. The generated query computes the average using `AVG(BUILDUPPLAYSPEED)`, which skips `NULL` values in both the sum and the count. The gold query instead computes `SUM(BUILDUPPLAYSPEED) / COUNT(ID)`, where `SUM` skips `NULL` values but `COUNT(ID)` counts all rows. SpotIt finds a counterexample where one `TEAM_ATTRIBUTES` row has `BUILDUPPLAYSPEED = NULL` and the other has `BUILDUPPLAYSPEED = -2`. Neither SpotIt$^+$-noV nor SpotIt$^+$ finds a counterexample because both extract a `NOT NULL` constraint on `BUILDUPPLAYSPEED`, which reflects the actual data and demonstrates that the counterexample that SpotIt finds would be trivial in a real-world scenario.

Generated Query P:

```
SELECT  AVG(BUILDUPPLAYSPEED)
FROM TEAM_ATTRIBUTES
JOIN TEAM
  ON TEAM_ATTRIBUTES.TEAM_API_ID
     = TEAM.TEAM_API_ID
WHERE TEAM.TEAM_LONG_NAME
  = 'HEART OF MIDLOTHIAN';
```

Gold Query Q:

```
SELECT  CAST(SUM(T2.BUILDUPPLAYSPEED)
 AS REAL) / COUNT(T2.ID)
FROM TEAM AS T1
INNER JOIN TEAM_ATTRIBUTES AS T2
  ON T1.TEAM_API_ID = T2.
       TEAM_API_ID
WHERE T1.TEAM_LONG_NAME
  = 'HEART OF MIDLOTHIAN';
```

```
--SpotIt Counterexample--
CREATE TABLE TEAM (
    ID INTEGER, TEAM_API_ID INTEGER, TEAM_FIFA_API_ID INTEGER,
    TEAM_LONG_NAME VARCHAR(20), TEAM_SHORT_NAME VARCHAR(20)
);
INSERT INTO TEAM VALUES (-2147483648, -2147483648, -2147483647, 'HEART
     OF MIDLOTHIAN', '2147483648');
INSERT INTO TEAM VALUES (-2147483647, -2147483648, -2147483648, 'HEART
     OF MIDLOTHIAN', '2147483648');
CREATE TABLE TEAM_ATTRIBUTES (
    ID INTEGER, TEAM_FIFA_API_ID INTEGER, TEAM_API_ID INTEGER,
    DATE VARCHAR(20), BUILDUPPLAYSPEED INTEGER, ...
);
INSERT INTO TEAM_ATTRIBUTES VALUES (-2147483648, -2147483647,
     -2147483648, '2147483648', NULL, ...);
INSERT INTO TEAM_ATTRIBUTES VALUES (-2147483647, -2147483647,
     -2147483648, '2147483648', -2, ...);
```

Fig. 3: Generated Query P, Gold Query Q, and SpotIt counterexample with bound $K = 2$ for NL Question: *"How much is the average build up play speed of the Heart of Midlothian team?"*.

Example D.2. Consider the natural language question: *"What is the most common bond type?"*. The generated query orders by `COUNT(BOND_TYPE)`, which skips `NULL` values, while the gold query orders by `COUNT(BOND_ID)`, which counts all rows in each group. SpotIt finds a counterexample where one bond has `BOND_TYPE = NULL`. Neither SpotIt$^+$-noV nor SpotIt$^+$ finds a counterexample because both extract a `NOT NULL` constraint on `BOND.BOND_TYPE` from the observed data, where every bond has a recorded type. Under this constrained domain, `COUNT(BOND_TYPE)` and `COUNT(BOND_ID)` produce identical counts within each group, making the queries equivalent.

Generated Query P:

```sql
SELECT BOND_TYPE
FROM BOND
GROUP BY BOND_TYPE
ORDER BY COUNT(BOND_TYPE) DESC
LIMIT 1;
```

Gold Query Q:

```sql
SELECT T.BOND_TYPE
FROM (
  SELECT
    BOND_TYPE,
    COUNT(BOND_ID)
  FROM BOND
  GROUP BY BOND_TYPE
  ORDER BY COUNT(BOND_ID) DESC
  LIMIT 1
) AS T;
```

```sql
-- SpotIt Counterexample--
CREATE TABLE BOND (
    BOND_ID VARCHAR(20), MOLECULE_ID VARCHAR(20), BOND_TYPE VARCHAR(20)
);
INSERT INTO BOND VALUES ('2147483648', '2147483649', NULL);
INSERT INTO BOND VALUES ('2147483649', '2147483648', '2147483649');
```

Fig. 4: Generated Query P, Gold Query Q, and SpotIt counterexample with bound $K = 2$ for NL Question: *"What is the most common bond type?"*.

Example D.3. Consider the natural language question: *"Which citizenship do the vast majority of the drivers hold?"*. The generated and gold queries differ in their `ORDER BY` clauses, with the generated query using `COUNT(NATIONALITY)` and the gold query using `COUNT(DRIVERID)`. Since `COUNT(column)` skips `NULL` values while `COUNT(DRIVERID)` counts all rows (as a primary key, `DRIVERID` is never `NULL`), a counterexample requires a driver with `NATIONALITY = NULL`. Both SpotIt and SpotIt$^+$ produce valid counterexamples exploiting this difference. However, SpotIt$^+$-noV extracts a `NOT NULL` constraint on `DRIVERS.NATIONALITY` from the observed data, where every driver has a recorded nationality. This constraint prevents Z3 from assigning `NULL` to any `NATIONALITY` value, making the two `COUNT` expressions equivalent under the constrained domain.

Generated Query P:

```sql
SELECT NATIONALITY
FROM DRIVERS
GROUP BY NATIONALITY
ORDER BY COUNT(NATIONALITY) DESC
LIMIT 1;
```

Gold Query Q:

```sql
SELECT NATIONALITY
FROM DRIVERS
GROUP BY NATIONALITY
ORDER BY COUNT(DRIVERID) DESC
LIMIT 1;
```

```sql
--SpotIt Counterexample--
CREATE TABLE DRIVERS (
    DRIVERID INTEGER,
    DRIVERREF VARCHAR(20),
    NUMBER INTEGER,
    CODE VARCHAR(20),
    FORENAME VARCHAR(20),
    SURNAME VARCHAR(20),
    DOB DATE,
    NATIONALITY VARCHAR(20),
    URL VARCHAR(20)
);
INSERT INTO DRIVERS VALUES
  (1, '2147483648', 0, '
       2147483648',
   '2147483648', '2147483648',
   '0001-01-01', NULL, '
        2147483648');
INSERT INTO DRIVERS VALUES
  (0, '2147483648', 0, '
       2147483648',
   '2147483648', '2147483648',
   '0001-01-01', '2147483649',
   '2147483648');
```

```sql
--SpotIt+ Counterexample--
CREATE TABLE DRIVERS (
    DRIVERID INTEGER,
    DRIVERREF VARCHAR(20),
    NUMBER INTEGER,
    CODE VARCHAR(20),
    FORENAME VARCHAR(20),
    SURNAME VARCHAR(20),
    DOB DATE,
    NATIONALITY VARCHAR(20),
    URL VARCHAR(20)
);
INSERT INTO DRIVERS VALUES
  (1, '2147483648', 0, '
       2147483648',
   '2147483648', '2147483648',
   '0001-01-01', NULL, '
        2147483648');
INSERT INTO DRIVERS VALUES
  (2, '2147483649', 0, '
       2147483648',
   '2147483648', '2147483649',
   '0001-01-02', '2147483649',
   '2147483648');
```

Example D.4. Consider the natural language question: *"List the last name of members with a major in environmental engineering and include its department and college name."*. The generated query is missing the `POSITION = 'MEMBER'` filter present in the gold query, so it returns all members majoring in Environmental Engineering regardless of their position. All three approaches find valid counterexamples containing members with positions other than `'MEMBER'` (such as `'Inactive'` and `'Secretary'`), which are returned by the generated query but filtered out by the gold query. The realism of the counterexamples varies significantly: SpotIt uses placeholder values like `'2147483648'` and `'8982140592376775526'` for positions, while SpotIt$^+$-noV and SpotIt$^+$ produce realistic categorical values such as `'Secretary'`, `'Medium'` for t-shirt sizes, and `'College of Natural Resources'` for the college name.

Generated Query P:

```
SELECT T1.LAST_NAME, T2.
    DEPARTMENT,
  T2.COLLEGE
FROM MEMBER AS T1
INNER JOIN MAJOR AS T2
  ON T1.LINK_TO_MAJOR = T2.
      MAJOR_ID
WHERE T2.MAJOR_NAME
  = 'ENVIRONMENTAL ENGINEERING';
```

Gold Query Q:

```
SELECT T2.LAST_NAME, T1.
    DEPARTMENT,
  T1.COLLEGE
FROM MAJOR AS T1
INNER JOIN MEMBER AS T2
  ON T1.MAJOR_ID = T2.
      LINK_TO_MAJOR
WHERE T2.POSITION = 'MEMBER'
  AND T1.MAJOR_NAME
  = 'ENVIRONMENTAL ENGINEERING';
```

```
--SpotIt Counterexample--
CREATE TABLE MAJOR (MAJOR_ID VARCHAR(20), MAJOR_NAME VARCHAR(20),
    DEPARTMENT VARCHAR(20), COLLEGE VARCHAR(20));
INSERT INTO MAJOR VALUES ('2147483649', 'ENVIRONMENTAL ENGINEERING', '
    2147483648', '2147483648');
INSERT INTO MAJOR VALUES (...);
CREATE TABLE MEMBER (MEMBER_ID VARCHAR(20), FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20), EMAIL VARCHAR(20),
    POSITION VARCHAR(20), T_SHIRT_SIZE VARCHAR(20), PHONE VARCHAR(20),
        ZIP INTEGER, LINK_TO_MAJOR VARCHAR(20));
INSERT INTO MEMBER VALUES ('2147483648', '2147483648', '2147483648', '
    2147483648', '2147483648', '2147483648', '2147483648', 1, '
    2147483648');
INSERT INTO MEMBER VALUES ('2147483649', '2147483648', '2147483648', '
    2147483648', '8982140592376775526', '2147483648', '2147483648', 1,
    '2147483649');
```

```
--SpotIt+-NoV Counterexample--
CREATE TABLE MAJOR (MAJOR_ID VARCHAR(20), MAJOR_NAME VARCHAR(20),
    DEPARTMENT VARCHAR(20), COLLEGE VARCHAR(20));
INSERT INTO MAJOR VALUES ('recdIBgeU38UbV2sy', 'ENVIRONMENTAL
    ENGINEERING', '2147483649', 'College of Natural Resources');
INSERT INTO MAJOR VALUES (...);
CREATE TABLE MEMBER (MEMBER_ID VARCHAR(20), FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20), EMAIL VARCHAR(20),
    POSITION VARCHAR(20), T_SHIRT_SIZE VARCHAR(20), PHONE VARCHAR(20),
        ZIP INTEGER, LINK_TO_MAJOR VARCHAR(20));
INSERT INTO MEMBER VALUES ('recf4UKTfipCzgcSA', '2147483648', '
    2147483649', '2147483649', 'Secretary', 'Large', '2147483648',
    1020, 'recxRBSgVYeSEGvyo');
INSERT INTO MEMBER VALUES ('recro8T1MPMwRadVH', '2147483649', '
    2147483648', '2147483648', 'Inactive', 'Small', '2147483649', 1021,
     'recdIBgeU38UbV2sy');
```

```
--SpotIt+ Counterexample--
CREATE TABLE MAJOR (MAJOR_ID VARCHAR(20), MAJOR_NAME VARCHAR(20),
    DEPARTMENT VARCHAR(20), COLLEGE VARCHAR(20));
INSERT INTO MAJOR VALUES ('2147483649', 'ENVIRONMENTAL ENGINEERING', '
    2147483649', '2147483649');
INSERT INTO MAJOR VALUES (...);
CREATE TABLE MEMBER (MEMBER_ID VARCHAR(20), FIRST_NAME VARCHAR(20),
    LAST_NAME VARCHAR(20), EMAIL VARCHAR(20),
    POSITION VARCHAR(20), T_SHIRT_SIZE VARCHAR(20), PHONE VARCHAR(20),
        ZIP INTEGER, LINK_TO_MAJOR VARCHAR(20));
INSERT INTO MEMBER VALUES ('2147483648', '2147483648', '2147483648', '
    2147483648', 'Inactive', 'X-Large', '2147483648', 0, '2147483649');
INSERT INTO MEMBER VALUES ('2147483649', '2147483648', '2147483648', '
    2147483648', 'Secretary', 'Medium', '2147483648', 1, '2147483649');
```

Fig. 5: Generated Query P, Gold Query Q, SpotIt, SpotIt$^+$-noV, and SpotIt$^+$ counterexamples with bound $K = 2$ for NL Question: *"List the last name of members with a major in environmental engineering and include its department and college name."*